

GuardianIQ

Policy Binding, Resolution & Runtime Enforcement Engine

End-to-End Engineering Implementation Specification

Developer-level specification for Database, Backend, Frontend, Runtime Interception, Approval Integration, Audit and QA

Field	Details
Prepared for	GuardianIQ Product, Architecture, Backend, Frontend, QA and DevOps Teams
Purpose	Define how policies are created, versioned, bound to models/agents/workflows/tools/data sources, resolved at runtime and enforced before governed actions execute.
Primary implementation stack	FastAPI / Python, PostgreSQL, React, GuardianIQ Governance Event Architecture
Document status	Engineering implementation specification
Core outcome	No governed AI action executes until applicable policy, relationship, permission and approval requirements have been resolved and an enforceable decision has been returned.

Source alignment: This specification extends the existing GuardianIQ Relationship Architecture and Step 4 Governance Event Architecture. Source-derived concepts include GOVERNED_BY relationships, policy_bindings, effective dates, policy lifecycle/events, correlation, agent tool-boundary checks, approval and audit events. Detailed rule-engine internals, APIs, UI behavior, schemas and enforcement gateway design below are implementation elaborations required to make those foundations build-ready.

Document Contents

1. Executive Summary
2. Scope, Boundaries and Design Principles
3. Source Architecture Alignment
4. Target Architecture
5. Policy Domain Model
6. Binding Model and Applicability
7. Policy Versioning and Lifecycle
8. Policy Rule Model and Decision Semantics
9. Database Specification
10. Backend Module Architecture
11. Policy Binding Service
12. Policy Resolution Engine
13. Policy Evaluation Engine
14. Runtime Enforcement Gateway
15. Agent, Model, Workflow, Tool and Data Enforcement
16. Approval, Exception and Override Integration
17. Governance Event and Audit Integration
18. API Design Standards
19. Detailed API Contracts
20. Frontend Architecture
21. Frontend Screen Specifications
22. Frontend Component Behavior
23. Security, RBAC, ABAC and Tenant Isolation
24. Caching, Performance and Resilience
25. Error Handling and Fail-Safe Behavior
26. Observability and Operational Monitoring
27. End-to-End Runtime Scenarios

- 28. Testing Strategy
- 29. Acceptance Test Catalogue
- 30. Development Sequence
- 31. Developer Task Breakdown
- 32. Deployment and Migration Plan
- 33. Definition of Done
- 34. Handover Checklist
- 35. Final End-to-End Technical Flow

- Appendix A. Enums
- Appendix B. Suggested Project Structure

1. Executive Summary

GuardianIQ requires policy governance to operate as an executable control plane, not as a document repository. A policy must be attachable to a governable object, resolvable in context, evaluated against runtime facts, and converted into an enforcement decision before an agent, workflow or user performs a controlled action.

The implementation target is a deterministic chain:

Canonical runtime chain: REQUEST → BUILD GOVERNANCE CONTEXT → RESOLVE BINDINGS → SELECT ACTIVE POLICY VERSIONS → EVALUATE RULES → COMBINE DECISIONS → ENFORCE → APPROVE/ESCALATE IF REQUIRED → EXECUTE OR BLOCK → WRITE IMMUTABLE EVENTS/EVIDENCE

- Policies can govern AI models, AI agents, workflows, tools/APIs, data sources, recommendations, actions, business units and other governable objects.
- Bindings support direct attachment, scoped attachment, mandatory policies, effective dates and priorities.
- Runtime resolution considers tenant, object relationships, business context, requested operation, data sensitivity, risk, execution mode and active policy version.
- Enforcement decisions are ALLOW, DENY, REQUIRE_APPROVAL, ESCALATE or ALLOW_WITH_OBLIGATIONS.
- Execution components must fail closed for configured high-risk operations when policy evaluation is unavailable or incomplete.
- Every evaluation and enforcement outcome produces traceable GuardianIQ governance events.

2. Scope, Boundaries and Design Principles

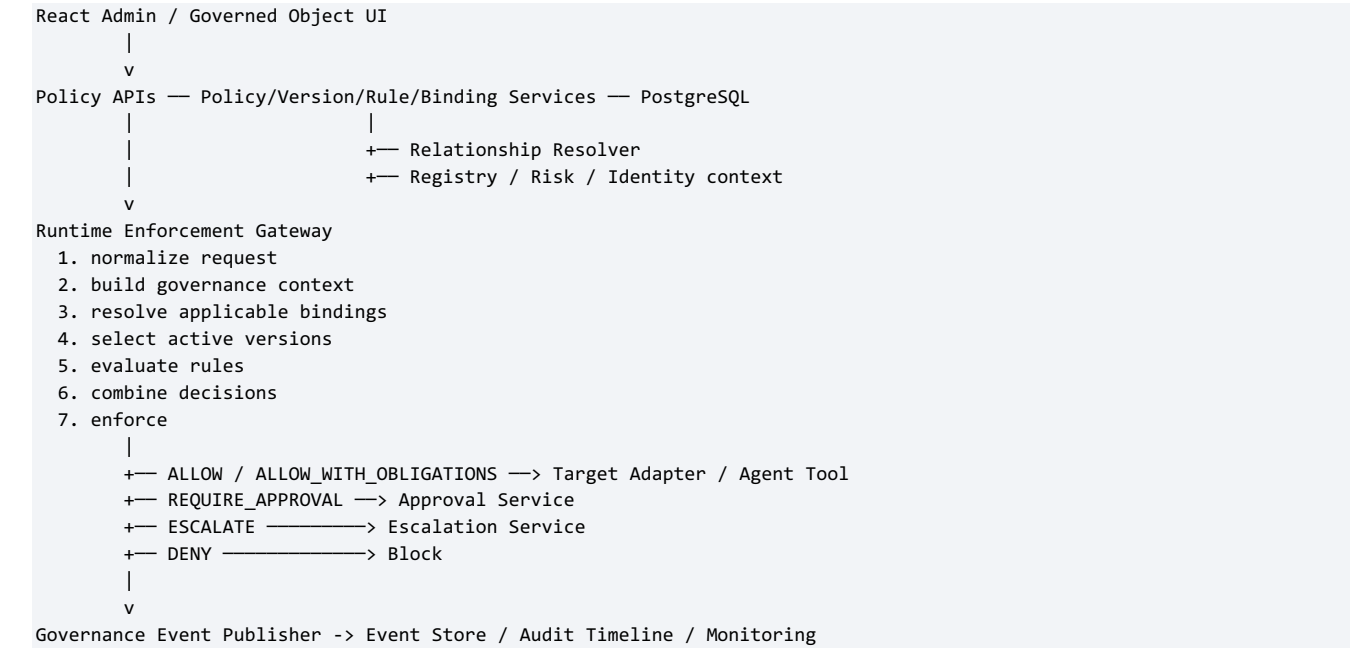
In Scope	Out of Scope / Later
Policy authoring metadata, versions and machine-evaluable rules	Natural-language-to-policy generation as an authoritative production control
Policy binding to governable objects and scopes	Replacing enterprise IAM/PAM products
Policy resolution, precedence and conflict handling	Deep external regulatory content management
Runtime evaluation and enforcement gateway	Full general-purpose BPM engine
Approval, exception and override handoff	Executing enterprise actions without the target system adapter
Frontend administration, simulation and enforcement history	Unreviewed autonomous policy changes
Audit/event integration, observability and tests	Storing secrets or raw sensitive payloads in policy/event records

- Backend is the source of truth; frontend visibility is never an enforcement control.
- Policy evaluation must be deterministic for the same policy version and normalized input.
- Historical evaluations retain the exact policy version and binding set used.
- Policy binding and policy evaluation are separate concepts: binding answers 'which policies apply'; evaluation answers 'what do they decide'.
- Relationships and permissions are evaluated before execution, not after the action.
- No production hard-delete of active policy history, bindings, evaluations or enforcement evidence.
- Tenant boundary is mandatory in every query, cache key and runtime request.

3. Source Architecture Alignment

Existing GuardianIQ foundation	Implementation implication
GOVERNED_BY relationship: any governable object → policy	Policy binding is a first-class relationship and must be queryable/resolvable.
policy_bindings: policy_id, target_type, target_id, scope, priority, mandatory, effective dates, status	Use as the base physical binding concept; extend for version strategy, conditions and inheritance.
Workflow policy-binding scenario	Workflow/schedule activation must be blocked when a mandatory governance binding is inactive.
Agent runtime boundary scenario	Resolve agent-tool-model relationships before invocation and fail fast when invalid.
POLICY_EVALUATED / POLICY_TRIGGERED	Emit for every authoritative runtime evaluation and triggered policy.
AGENT_ACTION_REQUESTED / TOOL_ACCESS_ATTEMPTED / AGENT_ACTION_BLOCKED	Use to prove pre-execution enforcement.
Approval and escalation events	REQUIRE_APPROVAL and ESCALATE decisions integrate with existing approval/escalation modules.
Append-only governance_events and correlation_id	Every runtime decision is reconstructable as one governed lifecycle.

4. Target Architecture



5. Policy Domain Model

Entity	Purpose	Key relationships
Policy	Stable business identity for a control	Has many versions; has many bindings
Policy Version	Immutable executable definition	Belongs to policy; contains rules

Policy Rule	Atomic condition + effect	Belongs to version
Policy Binding	Attaches policy to target/scope	Policy → model/agent/workflow/tool/data source/etc.
Policy Evaluation	One runtime evaluation record	References request, version(s), bindings, context hash
Rule Evaluation	Per-rule result	Belongs to policy evaluation
Enforcement Decision	Final combined decision	References evaluation and enforcement point
Policy Exception	Time-bounded approved deviation	Policy/binding/target/scope + approver
Approval Requirement	Human decision obligation	Created when result requires approval
Enforcement Obligation	Additional condition on allowed action	Mask, log, human-review, rate-limit, evidence, etc.

6. Binding Model and Applicability

A policy is not considered applicable merely because it exists. Applicability is determined by an ACTIVE binding whose tenant, target, scope, effective period and optional binding conditions match the runtime governance context.

Target Type	Example	Typical controls
AI_MODEL	MOD-LLM-001	approved provider, model risk tier, prohibited data, version restrictions
AI_AGENT	AGT-QUOTE-001	allowed tools, autonomy mode, max transaction value, human review
WORKFLOW	WF-RFQ-001	required approvals, evidence, schedule/run controls
TOOL	TOOL-SAP-PRICE	READ/WRITE/EXECUTE restrictions, environment, transaction boundary
DATA_SOURCE	DS-CUSTOMER-PII	classification, purpose limitation, masking, geographic restrictions
RECOMMENDATION	REC-1044	risk thresholds, approval and evidence
ACTION	ACT-9001	execution authority, monetary threshold, segregation of duties
BUSINESS_UNIT / DEPARTMENT	FINANCE	enterprise or domain-wide policy overlay

6.1 Binding Resolution Order

1. Load direct bindings for the runtime subject and requested target.
2. Load mandatory bindings inherited from workflow/business-unit/department/tenant scopes where configured.
3. Filter by tenant_id, status=ACTIVE and effective_from/effective_to.
4. Evaluate binding_scope and optional binding_condition_json.
5. Resolve the active policy version using the binding's version strategy.
6. Sort by mandatory flag, explicit priority and specificity.
7. Return the immutable resolved binding set with a resolution_hash.

6.2 Specificity and Precedence

Precedence	Example
1 - Explicit target binding	Policy bound directly to AGT-QUOTE-001
2 - Workflow scoped	Policy bound to WF-RFQ-001
3 - Department / BU scoped	Policy applies to COMMERCIAL
4 - Tenant-wide mandatory	Enterprise AI Acceptable Use
5 - Default platform control	GuardianIQ baseline safety control

Conflict rule: A lower-priority ALLOW must never override a higher-severity DENY. Mandatory policies cannot be bypassed by non-mandatory bindings. REQUIRE_APPROVAL dominates ALLOW. DENY dominates REQUIRE_APPROVAL unless an explicit, authorized exception is active.

7. Policy Versioning and Lifecycle

Object	States	Rules
Policy	DRAFT, ACTIVE, SUSPENDED, RETIRED	Stable identity; cannot be physically deleted after activation.
Policy Version	DRAFT, IN_REVIEW, APPROVED, ACTIVE, SUPERSEDED, RETIRED	Executable definition becomes immutable after activation.
Binding	DRAFT, ACTIVE, SUSPENDED, EXPIRED, REVOKED	Only ACTIVE and effective bindings participate in resolution.
Exception	REQUESTED, APPROVED, REJECTED, EXPIRED, REVOKED	Must have reason, scope, approver and expiration.

- New policy behavior requires a new policy version; do not mutate an ACTIVE version.
- Future evaluations use the newly ACTIVE version; historical evaluations retain the previous version.
- Binding version strategy: PINNED_VERSION or CURRENT_ACTIVE.
- Policy activation validates rule syntax, target compatibility, owner, effective dates and at least one test case.
- Suspending a mandatory policy must require elevated governance permission and generate an audit event.

8. Policy Rule Model and Decision Semantics

Field	Example
rule_code	QUOTE_HIGH_VALUE_APPROVAL
description	Quotes above threshold require Commercial Head approval
condition_json	{"all":[{"fact":"quote_value","op": ">","value":500000}]}
effect	REQUIRE_APPROVAL
severity	HIGH
obligations_json	{"approver_role":"COMMERCIAL_HEAD","sla_hours":8}
stop_processing	false
order_index	10

8.1 Supported operators - MVP

Category	Operators
Comparison	EQ, NE, GT, GTE, LT, LTE, IN, NOT_IN
Existence	EXISTS, NOT_EXISTS
String	CONTAINS, STARTS_WITH, MATCHES_PATTERN
Set	INTERSECTS, CONTAINS_ANY, CONTAINS_ALL
Context	IS_CLASSIFICATION, HAS_ROLE, HAS_PERMISSION, IS_ENVIRONMENT
Logical	ALL, ANY, NOT

8.2 Decision combination

Rule result(s)	Final decision
Any authoritative DENY	DENY
No DENY; one or more REQUIRE_APPROVAL	REQUIRE_APPROVAL
No DENY/approval; one or more ESCALATE	ESCALATE
Only ALLOW / obligations	ALLOW_WITH_OBLIGATIONS if obligations exist, else ALLOW
No applicable policy where policy is mandatory	DENY / CONFIGURATION_ERROR based on enforcement-point configuration
Evaluation engine unavailable for high-risk fail-closed point	DENY_TEMPORARY and operational alert

9. Database Specification

governance_policies

Column	Type / Constraint	Purpose
id	UUID PK	Policy identifier
tenant_id	UUID NOT NULL	Tenant boundary
policy_code	VARCHAR(100)	Unique per tenant
policy_name	VARCHAR(255)	Display name
policy_type	VARCHAR(80)	ACCESS / AGENT_BOUNDARY / APPROVAL / DATA / RISK / EXECUTION
description	TEXT	Business description
owner_user_id	UUID NULL	Policy owner
owner_group_id	UUID NULL	Owner group
status	VARCHAR(30)	DRAFT/ACTIVE/SUSPENDED/RETIRED
severity	VARCHAR(30)	LOW/MEDIUM/HIGH/CRITICAL
effective_from	TIMESTAMPTZ NULL	Policy effective start
effective_to	TIMESTAMPTZ NULL	Policy end
created_at	TIMESTAMPTZ	Created
updated_at	TIMESTAMPTZ	Updated

policy_versions

Column	Type / Constraint	Purpose
id	UUID PK	Version id
tenant_id	UUID NOT NULL	Tenant
policy_id	UUID FK	Parent policy
version_no	INTEGER	Monotonic version
version_label	VARCHAR(50)	v1, v2
status	VARCHAR(30)	Lifecycle
definition_json	JSONB	Normalized executable definition
definition_hash	VARCHAR(128)	Integrity/reproducibility
change_summary	TEXT	Why changed
approved_by	UUID NULL	Approver
approved_at	TIMESTAMPTZ NULL	Approval time
activated_at	TIMESTAMPTZ NULL	Activation time
created_at	TIMESTAMPTZ	Created

policy_rules

Column	Type / Constraint	Purpose
id	UUID PK	Rule id
tenant_id	UUID	Tenant
policy_version_id	UUID FK	Version
rule_code	VARCHAR(120)	Unique within version
order_index	INTEGER	Evaluation order
condition_json	JSONB	Condition AST
effect	VARCHAR(40)	Decision effect
severity	VARCHAR(30)	Rule severity
obligations_json	JSONB	Approver/mask/log/etc.
stop_processing	BOOLEAN	Short circuit
enabled	BOOLEAN	Included

policy_bindings

Column	Type / Constraint	Purpose
id	UUID PK	Binding id
tenant_id	UUID NOT NULL	Tenant boundary
policy_id	UUID FK	Policy
target_type	VARCHAR(100)	AI_AGENT/AI_MODEL/etc.
target_id	VARCHAR(150)	Target identifier

binding_scope	VARCHAR(255)	Workflow/BU/region/etc.
priority	INTEGER	Precedence
is_mandatory	BOOLEAN	Cannot be bypassed
version_strategy	VARCHAR(30)	CURRENT_ACTIVE/PINNED_VERSION
pinned_policy_version_id	UUID NULL	Pinned version
condition_json	JSONB	Optional applicability condition
effective_from	TIMESTAMPTZ	Start
effective_to	TIMESTAMPTZ NULL	End
status	VARCHAR(30)	Lifecycle
created_by	UUID	Actor
created_at	TIMESTAMPTZ	Created
updated_at	TIMESTAMPTZ	Updated

policy_evaluations

Column	Type / Constraint	Purpose
id	UUID PK	Evaluation id
tenant_id	UUID	Tenant
request_id	VARCHAR(150)	Runtime request
correlation_id	VARCHAR(150)	Lifecycle correlation
subject_type	VARCHAR(100)	Primary governed subject
subject_id	VARCHAR(150)	Subject id
operation	VARCHAR(150)	Requested operation
context_hash	VARCHAR(128)	Normalized input hash
resolved_bindings_json	JSONB	Exact binding/version set
decision	VARCHAR(40)	Final decision
decision_reason	TEXT	Explainable reason
risk_level	VARCHAR(30)	Resolved risk
evaluation_ms	INTEGER	Latency
created_at	TIMESTAMPTZ	Time

policy_rule_evaluations

Column	Type / Constraint	Purpose
id	UUID PK	Rule evaluation
policy_evaluation_id	UUID FK	Parent
policy_version_id	UUID	Evaluated version
rule_id	UUID	Rule
matched	BOOLEAN	Condition result
effect	VARCHAR(40)	Effect
facts_used_json	JSONB	Non-sensitive normalized facts/references
reason	TEXT	Explainability
evaluation_ms	INTEGER	Latency

policy_exceptions

Column	Type / Constraint	Purpose
id	UUID PK	Exception
tenant_id	UUID	Tenant
policy_id	UUID	Policy
binding_id	UUID NULL	Specific binding
target_type	VARCHAR(100)	Scope target
target_id	VARCHAR(150)	Target
operation	VARCHAR(150)	Optional operation
reason	TEXT	Business justification
status	VARCHAR(30)	Lifecycle
approved_by	UUID NULL	Approver
effective_from	TIMESTAMPTZ	Start
effective_to	TIMESTAMPTZ	Mandatory expiry
created_at	TIMESTAMPTZ	Created

enforcement_decisions

Column	Type / Constraint	Purpose
id	UUID PK	Enforcement record
tenant_id	UUID	Tenant
policy_evaluation_id	UUID FK	Evaluation
enforcement_point	VARCHAR(120)	AGENT_TOOL_GATEWAY/etc.
decision	VARCHAR(40)	Applied decision
obligations_json	JSONB	Applied obligations
approval_id	UUID NULL	Approval handoff
executed	BOOLEAN	Whether target execution occurred
execution_reference	VARCHAR(255)	External/action reference
created_at	TIMESTAMPTZ	Time

9.1 Required indexes and constraints

```

CREATE UNIQUE INDEX uq_policy_code_tenant
  ON governance_policies(tenant_id, policy_code);

CREATE UNIQUE INDEX uq_policy_version
  ON policy_versions(tenant_id, policy_id, version_no);

CREATE INDEX idx_binding_resolution
  ON policy_bindings(tenant_id, target_type, target_id, status, effective_from, effective_to);

CREATE INDEX idx_binding_policy
  ON policy_bindings(tenant_id, policy_id, status);

CREATE INDEX idx_policy_eval_subject
  ON policy_evaluations(tenant_id, subject_type, subject_id, created_at DESC);

CREATE INDEX idx_policy_eval_corr
  ON policy_evaluations(tenant_id, correlation_id);

-- Application + DB rule:
-- active policy versions are immutable; no hard delete after activation.

```

10. Backend Module Architecture

```
backend/modules/policy_engine/  
  models.py  
  schemas.py  
  enums.py  
  repository.py  
  policy_service.py  
  version_service.py  
  binding_service.py  
  binding_resolver.py  
  context_builder.py  
  rule_compiler.py  
  rule_evaluator.py  
  decision_combiner.py  
  exception_service.py  
  enforcement_service.py  
  router.py  
  validators.py  
  cache.py  
  
backend/modules/enforcement/  
  gateway.py  
  interceptors/  
    agent_action.py  
    tool_access.py  
    workflow_run.py  
    recommendation.py  
  adapters/  
    approval_adapter.py  
    escalation_adapter.py  
    action_adapter.py  
  
backend/modules/events/  
  ... existing Step 4 event publisher/outbox ...  
  
backend/shared/  
  actor_context.py  
  correlation.py  
  tenant_context.py  
  security_classification.py
```

11. Policy Binding Service

- Validate policy exists in same tenant and is not RETIRED.
- Validate target object exists and is governable.
- Reject cross-tenant bindings.
- Reject overlapping duplicate active binding for same policy/target/scope unless explicitly versioned.
- Validate effective_to > effective_from.
- Validate pinned version belongs to policy and is eligible.
- Emit binding lifecycle audit event for create/activate/suspend/revoke/expire.
- Invalidate resolver cache for affected policy and target.

```
def create_binding(cmd, actor):
    policy = policy_repo.require(cmd.policy_id, actor.tenant_id)
    target = registry.require_governable(cmd.target_type, cmd.target_id, actor.tenant_id)
    validate_scope(cmd.binding_scope, target)
    validate_dates(cmd.effective_from, cmd.effective_to)
    validate_no_conflicting_duplicate(cmd)
    binding = binding_repo.insert(cmd, actor)
    event_publisher.publish("POLICY_BINDING_CREATED", subject=binding, actor=actor)
    cache.invalidate_target(actor.tenant_id, cmd.target_type, cmd.target_id)
    return binding
```

12. Policy Resolution Engine

Resolution must return an explainable immutable set of bindings and policy versions. It must not evaluate business rules yet.

```
resolve(context):
    candidates = direct_bindings(context.subject)
    + target_bindings(context.tool/model/data_source)
    + workflow_bindings(context.workflow)
    + org_scope_bindings(context.department/business_unit)
    + tenant_mandatory_bindings(context.tenant)

    candidates = filter_active_effective_same_tenant(candidates, context.time)
    candidates = filter_binding_conditions(candidates, context)
    versions = select_policy_versions(candidates)
    exceptions = resolve_active_exceptions(candidates, context)

    return ResolvedPolicySet(
        bindings=sort_by_mandatory_priority_specificity(candidates),
        versions=versions,
        exceptions=exceptions,
        resolution_hash=hash(canonicalized_result)
    )
```

Resolver input	Required?
tenant_id	Yes
subject_type / subject_id	Yes
operation	Yes
workflow_id	When applicable
model_id / agent_id / tool_id / data_source_ids	When applicable
department / business_unit / region	When available
risk_context	When available
data_classification	When applicable
actor / execution_mode	Yes
occurred_at	Yes

13. Policy Evaluation Engine

8. Normalize runtime facts to a typed governance context.
9. Resolve policies and freeze the binding/version set for this evaluation.
10. Compile/validate condition AST; never execute arbitrary Python/JavaScript from policy JSON.
11. Evaluate rules against allow-listed facts only.
12. Capture matched rule, facts used by reference/value where safe, reason and latency.
13. Apply approved exceptions only to the policy/scope/operation explicitly covered.
14. Combine rule decisions using deterministic precedence.
15. Persist policy_evaluations and policy_rule_evaluations.
16. Publish POLICY_EVALUATED and POLICY_TRIGGERED events.
17. Return a signed/hashed evaluation response to the enforcement gateway where high assurance is required.

```
{
  "evaluation_id": "PEV-...",
  "correlation_id": "CORR-...",
  "decision": "REQUIRE_APPROVAL",
  "reason": "High-value ERP price update requires Commercial Head approval.",
  "policies": [
    { "policy_id": "POL-COMM-010", "version": 2, "binding_id": "...", "matched_rules": ["PRICE_WRITE_APPROVAL"] }
  ],
  "obligations": {
    "approver_role": "COMMERCIAL_HEAD",
    "sla_hours": 8,
    "evidence_required": true
  }
}
```

14. Runtime Enforcement Gateway

The gateway is the mandatory pre-execution control point for governed operations. Calling the Policy Evaluation API from the UI is not sufficient; execution adapters must invoke the gateway server-side immediately before the controlled operation.

Stage	Behavior
1. Request capture	Create request_id/correlation_id; emit AGENT_ACTION_REQUESTED or equivalent.
2. Relationship validation	Verify agent/model/tool/data/workflow relationships and permissions.
3. Context build	Load registry, risk, classification, ownership and execution context.
4. Policy resolve/evaluate	Obtain authoritative decision.
5. Enforce	ALLOW executes; DENY blocks; approval/escalation creates controlled handoff.
6. Post-decision evidence	Persist enforcement record and publish events.
7. Execution	Only after valid authorization token/decision; adapter invokes target.
8. Outcome	Publish ACTION_EXECUTED/FAILED or AGENT_ACTION_EXECUTED/FAILED.

```
async def enforce(req, actor):
    emit("AGENT_ACTION_REQUESTED", req)
    relationship_result = await relationship_guard.validate(req, actor)
    if not relationship_result.allowed:
        return block(req, "RELATIONSHIP_BOUNDARY")

    context = await context_builder.build(req, actor, relationship_result)
    evaluation = await policy_engine.evaluate(context)

    if evaluation.decision == "DENY":
        emit("AGENT_ACTION_BLOCKED", evaluation)
        emit("POLICY_VIOLATION_DETECTED", evaluation)
        raise PolicyDenied(evaluation)

    if evaluation.decision == "REQUIRE_APPROVAL":
        approval = approval_adapter.create(evaluation, req, actor)
        emit("APPROVAL_REQUESTED", approval)
        return PendingApproval(approval.id)

    if evaluation.decision == "ESCALATE":
        return escalation_adapter.create(evaluation, req, actor)

    apply_obligations(evaluation.obligations)
    result = await target_adapter.execute(req)
    emit("AGENT_ACTION_EXECUTED", result)
    return result
```

15. Agent, Model, Workflow, Tool and Data Enforcement

Enforcement point	Mandatory checks
-------------------	------------------

Agent action	Agent ACTIVE; owner resolved; execution mode allowed; policy set resolved; action within autonomy boundary.
Model invocation	Model/version approved; risk tier; permitted use case; provider/location constraints; prohibited data.
Tool/API access	USES_TOOL relationship; access_mode READ/WRITE/EXECUTE; requested operation; environment; approval requirement.
Workflow run	Workflow ACTIVE; mandatory GOVERNED_BY binding active; participants valid; schedule/context permitted.
Data access	USES_DATA_SOURCE; classification; purpose; field masking/tokenization obligation; region/tenant restrictions.
Recommendation → action	Policy/risk evaluation complete; evidence present; required approval current; no expired authorization.

15.1 Enforcement token / TOCTOU protection

- Approval/evaluation should not authorize a materially different later request.
- Generate an enforcement_context_hash from subject, operation, critical parameters, policy versions and approval.
- Execution adapter verifies the hash immediately before execution.
- If critical request facts changed after approval, re-evaluate and require re-approval when applicable.

16. Approval, Exception and Override Integration

Decision	System behavior
REQUIRE_APPROVAL	Create approval request with policy/version/rule, target, operation, context hash, required role/group and SLA.
APPROVED	Issue approval authorization tied to evaluation/context hash and expiry.
REJECTED	Block action; retain evaluation and reviewer rationale.
ESCALATE	Create escalation to resolved owner/group; do not execute unless policy explicitly permits pending escalation.
Exception requested	Create exception record; never silently bypass policy.
Exception approved	Apply only within target/operation/effective period; show in evaluation and audit.
Emergency override	Restricted role, mandatory reason, short expiry, elevated audit/alert; configurable DENY for policies that cannot be overridden.

17. Governance Event and Audit Integration

Event	When emitted
POLICY_CREATED / UPDATED / ACTIVATED / SUSPENDED / RETIRED	Policy lifecycle
POLICY_BINDING_CREATED / ACTIVATED / SUSPENDED / REVOKED / EXPIRED	Binding lifecycle - implementation extension
POLICY_EVALUATED	Every authoritative runtime evaluation
POLICY_TRIGGERED	One or more rule conditions trigger material effect
AGENT_ACTION_REQUESTED	Before agent action
TOOL_ACCESS_ATTEMPTED	Before tool/API invocation
AGENT_ACTION_VALIDATED	Control checks pass
AGENT_ACTION_BLOCKED / TOOL_ACCESS_DENIED	Pre-execution block
POLICY_VIOLATION_DETECTED	Attempt violates enforceable policy
APPROVAL_REQUESTED / GRANTED / REJECTED	Human control lifecycle
ESCALATION_TRIGGERED	Escalation required
ACTION_EXECUTED / ACTION_FAILED	Outcome
AUDIT_TIMELINE_UPDATED	Derived audit timeline refresh

Event discipline: The existing Step 4 foundation already requires append-only events, correlation IDs, policy context and audit reconstruction. New binding lifecycle event names above are implementation extensions and should be added to the event catalogue/schema registry before development.

18. API Design Standards

- Base path: /api/v1.
- All requests derive tenant from authenticated context; do not trust tenant_id supplied by browser.
- Standard response envelope: success, data, error, request_id, timestamp.
- Mutating endpoints require RBAC/ABAC and emit governance events.
- Evaluation endpoint for internal runtime must use service authentication and stricter rate/latency controls.
- Never return secrets, raw credentials or unrestricted sensitive context in evaluation explanations.

19. Detailed API Contracts

Method	Endpoint	Purpose
POST	/api/v1/policies	Create policy shell
GET	/api/v1/policies	Search/filter policies
GET	/api/v1/policies/{id}	Policy + versions + bindings summary
POST	/api/v1/policies/{id}/versions	Create draft version
POST	/api/v1/policy-versions/{id}/validate	Validate rules
POST	/api/v1/policy-versions/{id}/activate	Activate approved version
POST	/api/v1/policy-bindings	Bind policy to governable target
GET	/api/v1/policy-bindings	Search bindings
POST	/api/v1/policy-bindings/{id}/suspend	Suspend binding
POST	/api/v1/policy-bindings/{id}/revoke	Revoke binding
POST	/api/v1/policy-resolution/preview	Preview applicable policies without authoritative execution
POST	/api/v1/policy-evaluations	Authoritative internal evaluation
GET	/api/v1/policy-evaluations/{id}	Evaluation detail
POST	/api/v1/policy-simulations	Test policy against sample context
POST	/api/v1/policy-exceptions	Request exception
POST	/api/v1/policy-exceptions/{id}/approve	Approve exception
GET	/api/v1/objects/{type}/{id}/applicable-policies	Resolved policy view for object
GET	/api/v1/objects/{type}/{id}/enforcement-history	Policy/enforcement timeline

19.1 Create binding request

```
POST /api/v1/policy-bindings
{
  "policy_id": "POL-AI-GOV-001",
  "target_type": "AI_AGENT",
  "target_id": "AGT-QUOTE-001",
  "binding_scope": "WORKFLOW:WF-RFQ-001",
  "priority": 100,
  "is_mandatory": true,
  "version_strategy": "CURRENT_ACTIVE",
  "condition_json": {"environment": ["PRODUCTION"]},
  "effective_from": "2026-08-10T00:00:00Z",
  "effective_to": null
}
```

19.2 Authoritative evaluation request

```
POST /api/v1/policy-evaluations
{
  "subject": {"type": "AI_AGENT", "id": "AGT-QUOTE-001"},
  "operation": "ERP_PRICE_UPDATE",
  "workflow_id": "WF-RFQ-001",
  "tool": {"id": "TOOL-SAP-PRICE", "access_mode": "WRITE"},
  "model": {"id": "MOD-LLM-001", "version": "2026-07"},
  "facts": {
    "quote_value": 750000,
    "risk_score": 86,
    "data_classification": "CONFIDENTIAL",
    "environment": "PRODUCTION"
  },
  "correlation_id": "CORR-RFQ-1044"
}
```

20. Frontend Architecture

```
frontend/src/features/policies/
  pages/
    PolicyListPage.jsx
    PolicyDetailPage.jsx
    PolicyEditorPage.jsx
    BindingManagerPage.jsx
    PolicySimulationPage.jsx
    EvaluationHistoryPage.jsx
    ExceptionQueuePage.jsx
  components/
    PolicyHeader.jsx
    PolicyVersionTimeline.jsx
    RuleBuilder.jsx
    RuleRow.jsx
    BindingTable.jsx
    AttachPolicyDrawer.jsx
    ApplicablePoliciesPanel.jsx
    ResolutionTrace.jsx
    DecisionBadge.jsx
    EnforcementTimeline.jsx
    ExceptionDialog.jsx
  api/policyApi.js
  hooks/usePolicies.js
  hooks/useApplicablePolicies.js
  hooks/usePolicySimulation.js
```

21. Frontend Screen Specifications

Screen	Required capabilities	Primary roles
Policy Library	Search, filter, owner, type, severity, status, active version, binding count	Governance Admin, Risk, Auditor
Policy Detail	Metadata, versions, rules, bindings, trigger/evaluation history, audit	Governance Admin, Owner, Auditor
Policy Editor	Draft metadata, rule builder, validate, test cases, submit/activate	Policy Author, Governance Admin
Binding Manager	Attach policy to object/scope, priority, mandatory, effective dates, version strategy	Governance Admin
Object > Applicable Policies	Show direct/inherited policies, source binding, active version, priority, status	Object Owner, Admin, Auditor
Policy Simulation	Supply sample context, preview resolved bindings/rules/decision without execution	Policy Author, QA
Evaluation Detail	Context summary, binding resolution trace, matched rules, final decision, events	Admin, Auditor, Support

Enforcement History	Allow/block/approval/escalation timeline with correlation links	Owner, Auditor
Exception Queue	Request/review/approve/revoke time-bounded exceptions	Governance/Risk Approver
Agent/Model Detail Policy Tab	Policies, boundaries, last evaluation, violations, blocked actions	AI Owner, Governance

22. Frontend Component Behavior

- Attach Policy drawer must first choose target type/object, then policy, scope, mandatory flag, priority, version strategy and effective dates.
- UI must display whether a policy is DIRECT, INHERITED, TENANT_MANDATORY or DEFAULT.
- Inactive/expired binding must be visually distinct and never presented as currently enforcing.
- Rule builder validates operator against fact type and prevents arbitrary executable code.
- Simulation result must be clearly labelled NON-AUTHORITATIVE / NO EXECUTION.
- Evaluation detail must explain 'why this policy applied' and 'why this decision resulted'.
- DENY and REQUIRE_APPROVAL must show the specific policy version and matched rule.
- Frontend must not provide a bypass button. Exception/override must use governed workflows.

23. Security, RBAC, ABAC and Tenant Isolation

Capability	Admin	Policy Author	Object Owner	Approver	Auditor	Runtime Service
Create/edit draft policy	Yes	Yes	No	No	No	No
Activate policy	Yes	Conditional	No	No	No	No
Create binding	Yes	Conditional	Conditional	No	No	No
Suspend mandatory binding	Restricted	No	No	No	No	No
View evaluation	Yes	Scoped	Scoped	Scoped	Yes	Scoped
Approve exception	Restricted	No	No	Conditional	No	No
Authoritative evaluate	No	No	No	No	No	Yes
Execute governed action	Via normal app	Via normal app	Via normal app	Via normal app	No	Via enforcement gateway

- Enforce tenant_id in repositories, database query filters and cache keys.
- Service-to-service evaluation uses authenticated workload identity.
- Policy author cannot self-approve activation where segregation-of-duties is configured.
- Sensitive facts should be referenced or masked in logs/events; store only what is needed for reproducibility.
- Emergency override permission is separate from policy administration.

24. Caching, Performance and Resilience

Area	Recommendation
Binding cache	Key by tenant + target_type + target_id + relevant scope; invalidate on binding/policy/version change.
Policy version cache	Cache immutable ACTIVE version by version id/hash.
Evaluation latency	Target p95 < 150 ms for cached common path excluding external approval.
External dependencies	Context builder should prefer local registry/read models; avoid synchronous remote calls where possible.
Timeout	High-risk enforcement points fail closed on policy-engine timeout; low-risk behavior configurable.

Idempotency	request_id/idempotency_key prevents duplicate approval/evaluation side effects.
Horizontal scale	Evaluation service stateless except DB/cache; safe to scale replicas.

25. Error Handling and Fail-Safe Behavior

Scenario	Behavior
No active version for mandatory policy	Block and return POLICY_CONFIGURATION_ERROR; alert owner.
Binding references missing target/policy	Mark invalid; exclude from successful resolution; block if mandatory.
Policy rule syntax invalid	Cannot activate version.
Cross-tenant target/binding	403/validation failure; security event.
Evaluation timeout at critical enforcement point	Fail closed; no target execution.
Approval expired	Re-evaluate; do not execute.
Context changed after approval	Context hash mismatch; re-evaluate/re-approve.
Duplicate runtime request	Return existing idempotent result where safe.
Event publishing transient failure	Use transactional outbox; execution decision record remains recoverable.
Cache unavailable	Fallback to database resolution; do not bypass policy.

26. Observability and Operational Monitoring

Metric / Alert	Purpose
policy_evaluation_count by decision	Operational volume and policy impact
policy_evaluation_latency p50/p95/p99	Runtime performance
binding_resolution_latency	Resolver health
DENY / blocked action rate	Guardrail activity
REQUIRE_APPROVAL rate	Human review load
policy configuration errors	Broken mandatory policy/binding
policy engine timeout rate	Reliability
exception/override usage	Governance risk signal
cache hit ratio	Performance
top triggered policies/rules	Governance analytics
context hash mismatch after approval	TOCTOU / changed-request signal

27. End-to-End Runtime Scenarios

Scenario 1 - Agent updates ERP pricing

Context	Evaluation	Expected outcome
AGT-QUOTE-001 requests WRITE on TOOL-SAP-PRICE for ₹750,000.	Resolve agent + workflow + tool policies. Tool relationship allows READ only and high-value policy requires approval.	Block immediate execution. Emit TOOL_ACCESS_ATTEMPTED, POLICY_EVALUATED, AGENT_ACTION_BLOCKED, POLICY_VIOLATION_DETECTED; create approval/escalation as configured.

Scenario 2 - Workflow mandatory policy inactive

Context	Evaluation	Expected outcome
WF-001 schedule is activated.	WF-001 GOVERNED_BY POL-AI-GOV-001 is mandatory but binding is suspended.	Schedule activation fails before run; configuration error and audit event recorded.

Scenario 3 - Model handles restricted data

Context	Evaluation	Expected outcome
---------	------------	------------------

Agent proposes model MOD-EXT-001 with RESTRICTED customer data.	Data policy prohibits external model provider for RESTRICTED classification.	DENY before model invocation; no prompt/data sent to provider.
---	--	--

Scenario 4 - High-risk recommendation

Context	Evaluation	Expected outcome
Recommendation risk_score=91.	Policy resolves risk-board approval and evidence obligations.	REQUIRE_APPROVAL; recommendation cannot become executable action until approval and evidence are valid.

Scenario 5 - Policy v2 activation

Context	Evaluation	Expected outcome
POL-001 v2 replaces v1.	New evaluations select v2; existing audit records continue referencing v1.	Reproducible history with no retroactive mutation.

Scenario 6 - Approved exception

Context	Evaluation	Expected outcome
Temporary exception permits a specific agent operation for 2 hours.	Resolver finds approved exception matching policy/target/operation and effective window.	Decision records exception id and approver; after expiry normal policy blocks again.

28. Testing Strategy

Test category	Coverage
Unit	Rule operators, decision combination, date/effective filters, precedence, exception matching, context hashing
Repository	Tenant filtering, indexes, lifecycle updates, immutability
API	Policy/version/binding CRUD, simulation, evaluation, exception flows
Integration	Registry/relationship → resolver → evaluator → approval → action → events
Frontend	Rule builder, attach policy, applicable policy panel, simulation, evaluation detail
Security	Cross-tenant, unauthorized binding/activation, service auth, masking, override permission
Failure	DB/cache/event failure, timeouts, invalid mandatory binding, duplicate request
Performance	Resolution/evaluation latency under expected binding/rule volume
Audit	Correlation, policy version, binding, matched rule, actor and outcome reconstructable

29. Acceptance Test Catalogue

ID	Scenario	Expected result
PB-001	Bind active policy to AI agent	Binding ACTIVE and visible on agent policy tab; event generated.
PB-002	Attempt cross-tenant binding	Rejected.
PB-003	Resolve direct + tenant mandatory policies	Both returned with correct precedence/source.
PB-004	Expired binding	Not applicable.
PB-005	Pinned policy version	Resolver uses pinned version.
PB-006	CURRENT_ACTIVE after v2 activation	Future evaluation uses v2.
PB-007	DENY vs ALLOW conflict	DENY wins.
PB-008	REQUIRE_APPROVAL vs ALLOW	Approval wins.
PB-009	Agent READ-only tool attempts WRITE	Execution blocked before target call.
PB-010	Mandatory workflow policy suspended	Workflow activation/run blocked.
PB-011	Restricted data with prohibited model	Model invocation blocked; no data sent.
PB-012	Approval required	Approval created with policy/rule/context hash.
PB-013	Approval granted then request changed	Hash mismatch causes re-evaluation.

PB-014	Approved exception in window	Exception applied and audited.
PB-015	Exception expired	Normal policy enforced.
PB-016	Policy engine timeout critical action	Fail closed.
PB-017	Cache unavailable	DB fallback; same decision.
PB-018	Duplicate idempotency key	No duplicate approval/action.
PB-019	Evaluation audit reconstruction	Policy version, binding, rules and outcome visible.
PB-020	Policy author attempts unauthorized activation	403; audit/security event.
PB-021	Simulation	No production enforcement/action side effect.
PB-022	Rule syntax invalid	Version activation blocked.
PB-023	Mandatory binding references invalid policy version	Runtime blocked/configuration alert.
PB-024	Tool access allowed	Target adapter called only after ALLOW.
PB-025	Event correlation	Request→policy→approval→action events share correlation_id.

30. Development Sequence

- Freeze policy types, target types, decision enums and MVP rule operators.
- Create database migrations for policies, versions, rules, bindings, evaluations, rule evaluations, exceptions and enforcement decisions.
- Implement repositories with tenant isolation and lifecycle constraints.
- Implement policy/version service and activation validation.
- Implement binding service and binding lifecycle.
- Integrate with Relationship Architecture GOVERNED_BY and registry object resolution.
- Implement context builder and normalized fact model.
- Implement binding resolver with precedence/effective dates/version strategy.
- Implement safe rule evaluator and deterministic decision combiner.
- Implement exception resolution.
- Implement authoritative evaluation API and persistence.
- Implement enforcement gateway and agent/tool/workflow interceptors.
- Integrate approval/escalation adapters and context-hash authorization.
- Integrate Step 4 governance event publisher/outbox.
- Implement Policy Library, Editor and Binding Manager UI.
- Implement Applicable Policies panels on Agent/Model/Workflow/Tool/Data Source detail pages.
- Implement Simulation, Evaluation Detail, Enforcement History and Exception Queue.
- Add metrics, alerts, audit views and support diagnostics.
- Execute unit/API/integration/security/failure/performance tests.
- Run UAT scenarios and freeze v1 policy contract.

31. Developer Task Breakdown

Workstream	Tasks	Suggested owner
Database	Migrations, indexes, constraints, seed enums	Backend/Data
Policy Core	Policy/version/rule CRUD and validation	Backend
Binding	Binding CRUD, resolution, cache invalidation	Backend
Evaluation	Context builder, rule engine, combiner, explanations	Backend/AI Platform
Enforcement	Gateway, interceptors, adapters, fail-safe	Backend/Platform
Approval/Exception	Approval handoff, authorization hash, exceptions	Backend
Events/Audit	Event catalogue extension, publisher, timeline	Backend
Frontend Admin	Library/editor/binding/simulation	Frontend
Frontend Object UX	Applicable policies, history, violations	Frontend
QA	Automation, negative/security/UAT evidence	QA
DevOps	Metrics, alerts, service auth, secrets, scaling	DevOps

32. Deployment and Migration Plan

38. Deploy additive database schema with feature flags OFF.
39. Seed enums/reference policy types and enforcement points.
40. Deploy policy CRUD/version/binding APIs and admin UI.
41. Backfill existing GOVERNED_BY relationships into policy_bindings only after reconciliation; preserve source relationship ids.
42. Enable resolver in observe-only mode and compare decisions with expected scenarios.
43. Enable evaluation persistence/events without blocking execution.
44. Enable enforcement for one bounded pilot workflow/tool.
45. Enable REQUIRE_APPROVAL and DENY only after acceptance tests pass.
46. Expand enforcement points by risk tier and business unit.
47. Monitor blocked-action rate, latency, errors and overrides before enterprise rollout.

Rollout safety: Use OBSERVE_ONLY / ENFORCE modes per enforcement point during rollout. OBSERVE_ONLY may record what would have happened, but must never be represented to auditors as actual enforcement.

33. Definition of Done

- Policies are versioned and active versions are immutable.
- Policies can be bound to model, agent, workflow, tool, data source and organizational scopes.
- Applicable policies are resolved with tenant, scope, effective dates, priority and version strategy.
- Rule evaluation is deterministic and does not execute arbitrary code.
- Runtime gateway blocks or routes actions before target execution.
- Approval/exception authorization is bound to evaluation/context hash.
- All material policy/binding/evaluation/enforcement actions are evented and auditable.
- Frontend exposes policy attachment, applicability, simulation and enforcement history.
- RBAC/ABAC and tenant isolation are enforced server-side.
- Critical failure paths fail closed as configured.
- Acceptance catalogue passes and evidence is retained.

34. Handover Checklist

Item	Required
DB migrations reviewed and reversible	Yes
OpenAPI examples complete	Yes
Policy rule schema documented	Yes
Binding precedence documented in code/tests	Yes
Event catalogue/schema registry updated	Yes
Frontend routes/components merged	Yes
Unit/integration/security/performance tests passing	Yes
Pilot enforcement point demonstrated	Yes
Audit timeline demonstrated	Yes
Runbook for policy-engine outage/config error	Yes
Metrics/alerts dashboards configured	Yes
Product/Architecture/QA sign-off	Yes

35. Final End-to-End Technical Flow

- 1. Administrator creates POL-AI-GOV-001
- 2. Draft policy version v1 + executable rules created
- 3. Version validated, reviewed and activated
- 4. Policy bound:
 - AGT-QUOTE-001 GOVERNED_BY POL-AI-GOV-001
 - WF-RFQ-001 GOVERNED_BY POL-AI-GOV-001
- 5. Agent requests ERP_PRICE_UPDATE
- 6. Enforcement Gateway creates request/correlation context
- 7. Relationship guard validates:
 - agent -> workflow -> model -> tool -> data source
- 8. Policy Resolver finds direct/inherited/mandatory bindings
- 9. Resolver selects exact active/pinned policy versions
- 10. Evaluation Engine evaluates normalized runtime facts
- 11. Decision Combiner returns:
 - ALLOW | ALLOW_WITH_OBLIGATIONS | DENY |
 - REQUIRE_APPROVAL | ESCALATE
- 12. Gateway enforces the result BEFORE target execution
- 13. Approval/exception flow runs when required
- 14. Context hash is revalidated immediately before execution
- 15. Target adapter executes only when authorized
- 16. GuardianIQ writes immutable policy, agent, approval, action and audit events under the same correlation_id
- 17. Frontend shows Applicable Policies, Evaluation Detail, Enforcement History and Audit Timeline
- 18. Auditor can reconstruct:
 - who/what requested the action,
 - which policies and versions applied,
 - which rules matched,
 - why it was allowed/blocked,
 - who approved/overrode it,
 - and what outcome followed.

Final engineering principle: GuardianIQ policy enforcement is complete only when policy attachment, resolution, evaluation and execution control form one server-side chain. A policy displayed on an agent/model screen but not invoked at the execution boundary is governance metadata, not enforcement.

Appendix A. Core Enums

Enum	Values
PolicyStatus	DRAFT, ACTIVE, SUSPENDED, RETIRED
PolicyVersionStatus	DRAFT, IN_REVIEW, APPROVED, ACTIVE, SUPERSEDED, RETIRED
BindingStatus	DRAFT, ACTIVE, SUSPENDED, EXPIRED, REVOKED
Decision	ALLOW, ALLOW_WITH_OBLIGATIONS, DENY, REQUIRE_APPROVAL, ESCALATE
TargetType	AI_MODEL, AI_AGENT, WORKFLOW, TOOL, DATA_SOURCE, RECOMMENDATION, ACTION, BUSINESS_UNIT, DEPARTMENT
VersionStrategy	CURRENT_ACTIVE, PINNED_VERSION
Severity	LOW, MEDIUM, HIGH, CRITICAL
EnforcementMode	OBSERVE_ONLY, ENFORCE
AccessMode	READ, WRITE, EXECUTE
ExceptionStatus	REQUESTED, APPROVED, REJECTED, EXPIRED, REVOKED

Appendix B. Implementation Notes and Source Boundary

The existing GuardianIQ materials explicitly support: GOVERNED_BY as the policy applicability relationship; policy_bindings with target/scope/priority/mandatory/effective-date semantics; policy version upgrade behavior for future evaluations; runtime boundary checks; agent/tool access violations; policy evaluation/trigger events; approval/escalation; append-only event history; and tenant/RBAC controls.

The following are implementation elaborations introduced by this specification to make the capability developer-ready: policy_versions and policy_rules physical structures; safe condition AST/operators; decision-combination precedence; enforcement gateway; enforcement context hash; observe-only rollout; detailed APIs; detailed frontend screens/components; exception data model; caching targets; latency targets; and the expanded acceptance-test catalogue. These should be reviewed and approved as GuardianIQ product architecture decisions before production build.